Homework 2

sli602

October 2026

All the codes are provided in github repo: https://github.com/SiyuLi915/CS521

1 Problem 1

(a) The exact sum, and hence the most precise complex interval, is

$$I_1 + I_2 = [a_1 + a_2, b_1 + b_2] + i[c_1 + c_2, d_1 + d_2].$$

This follows because the real and imaginary coordinates vary independently.

(b) For real intervals $[u, v]$ and $[s, t]$, define

$$m(u, v; s, t) = \min\{us, ut, vs, vt\},$$
$$M(u, v; s, t) = \max\{us, ut, vs, vt\}.$$

These are the minimum and maximum of the product of one number from each interval: the extrema of the bilinear function occur at endpoint pairs.

Expanding the complex product gives

$$z_1 z_2 = (x_1 x_2 - y_1 y_2) + i(x_1 y_2 + y_1 x_2).$$

Define

$$r_{\min} = m(a_1, b_1; a_2, b_2) - M(c_1, d_1; c_2, d_2),$$
$$r_{\max} = M(a_1, b_1; a_2, b_2) - m(c_1, d_1; c_2, d_2),$$
$$s_{\min} = m(a_1, b_1; c_2, d_2) + m(c_1, d_1; a_2, b_2),$$
$$s_{\max} = M(a_1, b_1; c_2, d_2) + M(c_1, d_1; a_2, b_2).$$

Then the most precise complex interval containing $I_1 \cdot I_2$ is

$$J_\times = [r_{\min}, r_{\max}] + i[s_{\min}, s_{\max}].$$

Each bound is attained by some choice of $z_1 \in I_1$, $z_2 \in I_2$. So no smaller rectangle contains $\{z_1 z_2\}$, and this is the most precise sound complex interval.

2 Problem 2

(a) The input coordinates are

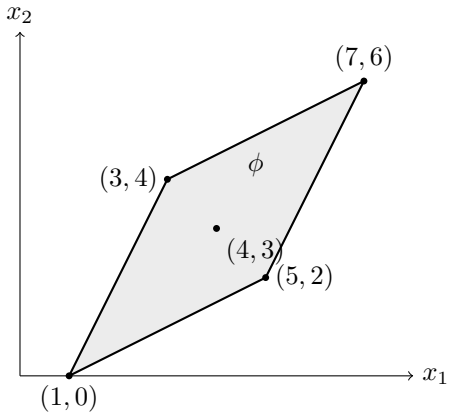
$$\hat{x}_1 = 4 + 2\epsilon_1 + \epsilon_2, \qquad \hat{x}_2 = 3 + \epsilon_1 + 2\epsilon_2.$$

Taking $(\epsilon_1, \epsilon_2) \in \{-1, 1\}^2$ gives the vertices.

$(\varepsilon_1, \varepsilon_2)$	(x_1, x_2)
$(-1, -1)$	$(1, 0)$
$(1, -1)$	$(5, 2)$
$(1, 1)$	$(7, 6)$
$(-1, 1)$	$(3, 4)$

Thus ϕ is the parallelogram centered at $(4, 3)$:

$$\phi = \mathrm{conv}\{(1, 0), (5, 2), (7, 6), (3, 4)\}.$$



(b) Yes. The affine transformer is exact:

$$\begin{pmatrix} \hat{x}_3 \\ \hat{x}_4 \end{pmatrix} = \begin{pmatrix} 1 & 2 \\ -1 & 1 \end{pmatrix} \begin{pmatrix} \hat{x}_1 \\ \hat{x}_2 \end{pmatrix} = \begin{pmatrix} 10 \\ -1 \end{pmatrix} + \begin{pmatrix} 4 \\ -1 \end{pmatrix} \epsilon_1 + \begin{pmatrix} 5 \\ 1 \end{pmatrix} \epsilon_2.$$

ReLU transformer. For $y = \mathrm{ReLU}(\hat{x})$ with $\hat{x} \in [l, u]$:

$$\hat{y} = \begin{cases} 0, & u \le 0, \\ \hat{x}, & l \ge 0, \\ \lambda \hat{x} + \mu + \mu\, \varepsilon_{\mathrm{new}}, & l < 0 < u, \end{cases} \qquad \lambda = \frac{u}{u - l}, \quad \mu = -\frac{\lambda l}{2},$$

where $\varepsilon_{\mathrm{new}} \in [-1, 1]$ is a fresh noise symbol. Consequently,

$$\hat{x}_3 \in [1, 19], \qquad \hat{x}_4 \in [-3, 1].$$

Since $\hat{x}_3$ is always positive, its ReLU transformer is the identity:

$$\hat{x}_5 = \hat{x}_3 = 10 + 4\epsilon_1 + 5\epsilon_2.$$

For $\hat{x}_4$, introduce a fresh noise symbol $\epsilon_3 \in [-1, 1]$ with $\lambda = \frac{1}{4}$ and $\mu = \frac{3}{8}$ and use

$$\hat{x}_6 = \frac{1}{4}\hat{x}_4 + \frac{3}{8} + \frac{3}{8}\epsilon_3 = \frac{1}{8} - \frac{1}{4}\epsilon_1 + \frac{1}{4}\epsilon_2 + \frac{3}{8}\epsilon_3.$$

This transformer is sound because

$$0 \leq \mathrm{ReLU}(t) - \frac{1}{4}t \leq \frac{3}{4} \qquad \text{for every } t \in [-3, 1].$$

Thus every concrete ReLU output is represented by some $\epsilon_3 \in [-1, 1]$.

Finally, throughout the resulting output zonotope,

$$\begin{aligned} \hat{x}_5 - \hat{x}_6 &= \frac{79}{8} + \frac{17}{4}\epsilon_1 + \frac{19}{4}\epsilon_2 - \frac{3}{8}\epsilon_3 \\ &\geq \frac{79}{8} - \frac{17}{4} - \frac{19}{4} - \frac{3}{8} = \frac{1}{2} > 0. \end{aligned}$$

By soundness, this bound also holds for every concrete output. Therefore,

$$x_5 \geq x_6 \quad \text{for all } (x_1, x_2) \in \phi.$$

3 Problem 3

(a) Assume $a_i \le b_i$ for $i = 1, 2$. Introduce $z \in \{0, 1\}$ and define

$$M_1 = \max\{0, b_2 - a_1\}, \qquad M_2 = \max\{0, b_1 - a_2\}.$$

An exact MILP encoding of $y = \max(x_1, x_2)$ is

$$\begin{aligned} & a_i \le x_i \le b_i && (i = 1, 2), \\ & y \ge x_1, \qquad y \ge x_2, \\ & y \le x_1 + M_1(1 - z), \\ & y \le x_2 + M_2 z, \\ & z \in \{0, 1\}. \end{aligned}$$

If $z = 1$, these constraints force $y = x_1 \ge x_2$; if $z = 0$, they force $y = x_2 \ge x_1$. Conversely, choosing a maximizing input is always feasible because

$$x_2 - x_1 \le M_1, \qquad x_1 - x_2 \le M_2.$$

Thus the encoding is exact, including ties.

(b) From $x_1, x_2 \in [0, 1]$, the first affine layer gives

$$x_3 \in [0, 2], \quad x_4 \in [-2, -1], \quad x_5 \in [-1, 1], \quad x_6 \in [0, 1].$$

Since $x_3 - x_4 = x_2 + 2 > 0$, the first maxpool simplifies exactly to $x_7 = x_3$. For $x_8 = \max(x_5, x_6)$, part (a) gives $M_1 = 2$ and $M_2 = 1$.

Hence the network is encoded by

$$\begin{aligned} & 0 \le x_1 \le 1, \qquad 0 \le x_2 \le 1, \\ & x_3 = x_1 + x_2, \qquad x_4 = x_1 - 2, \\ & x_5 = x_1 - x_2, \qquad x_6 = x_2, \\ & x_7 = x_3, \\ & x_8 \ge x_5, \qquad x_8 \ge x_6, \\ & x_8 \le x_5 + 2(1 - z), \qquad x_8 \le x_6 + z, \\ & x_9 = x_7, \qquad x_{10} = -x_7 + x_8 - \frac{1}{2}, \\ & z \in \{0, 1\}. \end{aligned}$$

Equivalently, setting $t = x_5 - x_6 \in [-2, 1]$ and $r = x_8 - x_6$ turns the maxpool constraints into the ReLU encoding

$$r \ge 0, \qquad r \ge t, \qquad r \le t + 2(1 - z), \qquad r \le z.$$

To verify the property, minimize

$$\Delta = x_9 - x_{10} = 2(x_1 + x_2) - x_8 + \frac{1}{2}$$

subject to the MILP constraints. The two binary cases give

$$\Delta = \begin{cases} x_1 + 3x_2 + \frac{1}{2}, & z = 1 \quad (x_8 = x_5), \\ 2x_1 + x_2 + \frac{1}{2}, & z = 0 \quad (x_8 = x_6). \end{cases}$$

Since $x_1, x_2 \geq 0$, both cases satisfy $\Delta \geq \frac{1}{2}$. Equality is attained at $(x_1, x_2) = (0, 0)$. Therefore,

$$\min \Delta = \frac{1}{2} > 0, \qquad x_9 > x_{10} \quad \text{for all } (x_1, x_2) \in [0, 1]^2.$$

Thus MILP analysis proves the property; equivalently, the network constraints together with $x_9 \leq x_{10}$ are infeasible.

4 Problem 4

Colab's link: https://colab.research.google.com/drive/1cMU6wZuTUsxx7Q7bxLA0pBMB4s8Re6mw?usp=sharing

5 Problem 5

(a) First, I kept DeepPoly's shape and changed only the two transformers (`mycert.cf`) but the certifier is less precise than DeepPoly, so I changed the shape to make it better. My final design is the following:

Shape. I add a second polyhedral lower bound, `La`:

```
(Float l, Float u, PolyExp L, PolyExp U, PolyExp La)
```

with the extra constraint `curr[La] <= curr`.

ReLU. When the input crosses zero, both 0 and x are sound lower bounds. `L` keeps DeepPoly's area-heuristic choice, and `La` stores the other choice. Stable neurons use the same bound in `L` and `La`. The remaining ReLU computations are unchanged.

Affine. I backsubstitute twice, once using `L` and once using `La` for nonnegative coefficients; both passes use `U` for negative coefficients. I set

$$l = \max\{l_L, l_{L^a}\}.$$

Both results are sound lower bounds, so their maximum is sound and no looser than the `L`-based pass on the same stored state. The computation of `u` is unchanged, while `L`, `U`, and `La` are all set to the exact affine expression.

Why useful. DeepPoly must commit to one lower bound (0 or x) for each unstable ReLU, and the area heuristic only looks at that one neuron. The choice that gives the tightest bound after backsubstitution can be different. `deeppoly2l` keeps both choices and, at each Affine neuron, uses the larger of the two lower bounds. Given the same polyhedral bounds from the earlier layers, this lower bound is never looser than DeepPoly's. The cost is one additional backsubstitution pass for each Affine layer.

(b) Output:

```
Proved Affine
Proved Relu
Provesound result: None
```

(c) Experiments

	DeepPoly		`deeppoly2l`	
ϵ	Verified	Time/image (s)	Verified	Time/image (s)
0.005	99%	0.213	99%	0.446
0.01	94%	0.273	94%	0.446
0.02	83%	0.275	83%	0.442
0.03	49%	0.287	49%	0.366
0.05	2%	0.319	2%	0.363

Comparison. The two certifiers verify the same percentage of images for every ϵ. `deeppoly2l` takes about 1.1–2.1× as long for each image, because each Affine layer has one more backsubstitution pass.

Why the results are the same. I compared the final bounds of both certifiers on each image. The new certifier does tighten some bounds, but the gain is very small. The largest increase of a bound on $\text{logit}_y - \text{logit}_j$ is 2.3×10^{-3} at $\epsilon = 0.02$ and 7.5×10^{-5} at $\epsilon = 0.03$. In comparison, the closest unverified image at $\epsilon = 0.02$ has a margin of -0.071. So no result changes.

Code. `https://github.com/SiyuLi915/CS521/tree/main/hw2/q5/deeppoly2l.cf`

(d)

```
func slope_hi(Neuron n) = (1.0 - n[l]) / (n[u] - n[l]);
func ht_lower_hi(Neuron n) = (slope_hi(n) * n) + (n[l] - (slope_hi(n) * n[l]));
func ht_upper_hi(Neuron n) = n[l] + n[u] <= 2.0 ? n : 1.0;
func slope_lo(Neuron n) = (n[u] + 1.0) / (n[u] - n[l]);
func ht_lower_lo(Neuron n) = n[l] + n[u] >= (0.0 - 2.0) ? n : (0.0 - 1.0);
func ht_upper_lo(Neuron n) = (slope_lo(n) * n) - ((slope_lo(n) * n[l]) + 1.0);
func ht_lower_both(Neuron n) = n[l] + n[u] >= 0.0 ?
    ((2.0 / (n[u] + 1.0)) * n) + ((2.0 / (n[u] + 1.0)) - 1.0) : (0.0 - 1.0);
func ht_upper_both(Neuron n) = n[l] + n[u] <= 0.0 ?
    ((2.0 / (1.0 - n[l])) * n) + (1.0 - (2.0 / (1.0 - n[l]))) : 1.0;

HardTanh -> (prev[u] <= (0.0 - 1.0)) ?
  ((0.0 - 1.0), (0.0 - 1.0), (0.0 - 1.0), (0.0 - 1.0)) :
  ((prev[l] >= 1.0) ? (1.0, 1.0, 1.0, 1.0) :
  (((prev[l] >= (0.0 - 1.0)) and (prev[u] <= 1.0)) ?
      (prev[l], prev[u], prev, prev) :
  ((prev[l] >= (0.0 - 1.0)) ?
      (prev[l], 1.0, ht_lower_hi(prev), ht_upper_hi(prev)) :
  ((prev[u] <= 1.0) ?
      ((0.0 - 1.0), prev[u], ht_lower_lo(prev), ht_upper_lo(prev)) :
      ((0.0 - 1.0), 1.0, ht_lower_both(prev), ht_upper_both(prev))))));
```

Soundness check. Output:

```
Proved Affine
Proved HardTanh
Provesound result: None
```

`https://github.com/SiyuLi915/CS521/tree/main/hw2/q5/deeppoly_hardtanh.cf`